
BlueLine

Universal Security & Reliability Assessment Platform

Assume nothing. Test aggressively. Prove what you find.

Project Type: Offline-first multi-technique security & reliability assessment platform

Author: Amirsam Eftekharia

Repository: <https://github.com/AmirSam-Eftekhari/BlueLine>

Version: 1.0.0

Document Date: September 2026

Document Type: Project Proposal / Technical Project Report

This document describes the actual implementation present in the BlueLine repository. Features are classified as implemented, partially supported, or experimental based on source-code evidence. No functionality is claimed beyond what the code implements.

1. Table of Contents

- 2. Abstract
- 3. Introduction
- 4. Problem Statement
- 5. Project Objectives
- 6. Scope
- 7. System Architecture
- 8. Target Discovery and Input Handling
- 9. Static Analysis
- 10. Multi-Language Support
- 11. Dynamic Analysis
- 12. Behavior-Guided Generational Fuzzing
- 13. Finding Correlation and Validation
- 14. Risk Assessment
- 15. Coverage
- 16. Persistence and History
- 17. Logging and Observability
- 18. Reporting System
- 19. User Interface
- 20. Privacy and Offline-First Architecture
- 21. Error Handling and Recovery
- 22. Testing and Verification
- 23. Build and Deployment
- 24. Security Considerations
- 25. Technical Challenges
- 26. Design Decisions and Trade-offs
- 27. Current Limitations
- 28. Future Development
- 29. Conclusion
- 30. Technical Appendix
- 31. References

2. Abstract

BlueLine is an offline-first security and reliability assessment platform that analyzes heterogeneous targets—source repositories, scripts, executables, and live HTTP endpoints—entirely on the user’s machine. It discovers the nature of a target, selects only applicable analyzers, runs static analysis (with the deepest implementation for Python via AST), dependency and configuration checks, controlled dynamic execution, behavior-guided generational fuzzing, binary metadata extraction, and limited read-only web/API checks.

Findings are produced under a unified schema that separates severity, confidence, validation status, risk score, and coverage. A correlation engine merges corroborating findings; a transparent risk engine computes scores from explicit factors; a coverage engine reports what was actually executed versus skipped or failed. Results persist in a local SQLite store, support history and regression comparison, and export to HTML, JSON, SARIF 2.1.0, CSV, and (when available) PDF. The architecture deliberately isolates analyzer failures, binds the local Flask UI only to 127.0.0.1, and keeps target data local.

The core philosophy—“Assume nothing. Test aggressively. Prove what you find.”—is reflected in explicit capability levels (FULL / PARTIAL / EXPERIMENTAL / UNSUPPORTED), validation statuses (CONFIRMED / PROBABLE / SUSPECTED / ...), and coverage reporting that never equates “nothing found” with “safe.”

3. Introduction

Security assessment of real software is rarely a single-technique problem. Purely signature- or pattern-based static scanners produce high false-positive rates and lack runtime evidence. Pure dynamic testing or fuzzing can miss deep structural issues and is expensive. Binary and configuration analysis address different surfaces. Heterogeneous projects (multiple languages, build systems, binaries, and HTTP interfaces) require honest capability declaration rather than silent skipping or fabricated coverage.

BlueLine addresses this by treating assessment as a pipeline of discovery → capability planning → multi-technique execution → correlation → risk and coverage computation → persistence and reporting. It is designed as a local tool: target data remains on the machine; the API server binds only to localhost; logs intentionally omit evidence snippets that may contain secrets or source. Engineering choices emphasize modular analyzers behind a common interface, isolation of stage failures, transparent risk formulas, and reproducible fuzzing campaigns (fixed mutator seed by default).

The project is implemented primarily in Python 3, with a Flask-based local web UI, CLI entry points, and optional PyInstaller packaging for Windows.

4. Problem Statement

Key technical problems the repository directly confronts:

- Static analysis alone yields many unvalidated heuristics; confidence and validation must be first-class.
- Different techniques may report the same underlying issue; without correlation, reports become noisy duplicates.
- Capability varies sharply by language and target type; claiming uniform support is incorrect. The system must declare FULL_SUPPORT, PARTIAL_SUPPORT, EXPERIMENTAL, or UNSUPPORTED and surface unsupported surfaces.
- Dynamic and fuzzing runs must be resource-bounded and isolated so a single bad target does not crash the whole assessment.
- “No findings” is not equivalent to safety; coverage of what was actually tested (and what failed or was skipped) must be reported.

- History and reproducibility matter for regression comparison; results must be persisted with stable fingerprints.
- Offline/local processing is required for privacy-sensitive targets; network exposure of the UI and target data must be minimized by design.

The implementation does not solve every instance of these problems (e.g., interprocedural analysis is limited; most non-Python languages are pattern-oriented rather than full AST), but it structures the system around explicit handling of them.

5. Project Objectives

5.1 Security / Static Analysis

- Provide an AST-based rule engine for Python with same-function taint tracking for selected data-flow issues (notably SQL).
- Provide partial static analyzers for additional languages.
- Detect common insecure patterns, secrets, weak cryptography, and misconfigurations.

5.2 Reliability / Dynamic Analysis

- Execute candidate entry points under timeouts, process isolation, and optional activity monitoring (file/network via psutil).
- Classify outcomes (crash, timeout, etc.) and convert them into findings.

5.3 Fuzzing

- Implement behavior-guided generational mutation that promotes inputs producing novel observable behavior, with reproducible campaigns and anomaly deduplication.

5.4 Finding Validation and Correlation

- Maintain validation status (CONFIRMED when dynamically/fuzz-reproduced, etc.).
- Merge findings sharing fingerprints, elevating status when corroboration exists.

5.5 Risk Assessment

- Compute a transparent continuous risk score from impact, exploitability, exposure, confidence, and reproducibility factors, then map to severity bands.

5.6 Coverage, Reporting, Privacy, Usability

- Report per-stage and overall assessment coverage based on actual stage outcomes.
- Export HTML, JSON, SARIF 2.1.0, CSV, and PDF (PDF may require additional runtime support).
- Keep target analysis and storage local; bind UI/API to 127.0.0.1; limit sensitive data in logs.
- Local web UI + CLI; scan profiles (quick / standard / deep / maximum); history and compare APIs.

6. Scope

6.1 In Scope (Implemented)

- Target discovery and profiling.
- Modular analyzer registry and orchestrator with failure isolation.
- Python AST static analysis with multiple rules and same-function taint for SQL-related cases.
- Partial static analyzers for JS/TS, Java, Go, Ruby, PHP, Rust, Kotlin, Swift, Scala; experimental C/C++.
- Dependency and configuration analyzers.

- Binary analysis (ELF parser via struct; PE metadata; Mach-O string extraction).
- Read-only web/API checks (GET/HEAD/OPTIONS).
- Dynamic runner with limits and optional activity monitoring.
- Behavior-guided generational fuzzing.
- Correlation, risk engine, coverage engine.
- SQLite persistence with history and compare.
- Structured multi-stream JSONL logging.
- HTML / JSON / SARIF 2.1 / CSV / PDF reporting.
- Flask local UI/API (127.0.0.1:8642).
- CLI and desktop launcher; PyInstaller packaging artifacts for Windows.
- Substantial automated test suite under tests/.

6.2 Partially Supported

- Most non-Python languages (pattern/heuristic rather than deep AST/taint).
- PE (no full import table); Mach-O (primarily strings).
- Web/API (read-only header/CORS/banner checks, not a full active scanner).
- Dynamic/fuzzing (requires detectable CLI/entry points; stdin-driven).

6.3 Experimental

- C/C++ static analysis.

6.4 Out of Scope / Not Implemented

- Interprocedural or whole-program data-flow for non-Python (and limited even for Python).
- Full active web exploitation (auth, injection, state-changing requests).
- Code-coverage-guided fuzzing (no compile-time instrumentation of arbitrary targets).
- Cloud/remote analysis or upload of target data.
- Full PE import/dependency graphing, advanced binary decompilation, or symbolic execution.
- Guaranteed multi-platform sandboxing beyond process-group / resource limits.
- Production-grade distributed orchestration.

7. System Architecture

Major components reconstructed from the repository source layout:

- **app/targets/discovery.py** — TargetDiscovery
- **app/analyzers/** — Analyzer base, concrete analyzers, and registry
- **app/core/orchestrator.py** — ScanOrchestrator
- **app/core/models.py** — unified Finding, TargetProfile, ScanConfig, ScanResult
- **app/core/correlation.py, risk.py, coverage.py**
- **app/runtime/dynamic.py** — DynamicRunner
- **app/fuzzing/** — FuzzingEngine + Mutator
- **app/persistence/store.py** — SQLite ScanStore
- **app/reporting/** — HTML, JSON, SARIF, CSV, PDF generators
- **app/core/logging_setup.py** — multi-stream structured logs
- **app/api_server.py** — Flask local API + static UI serving
- **app/cli.py, desktop_launcher.py; ui/; packaging/**

Pipeline (actual orchestrator flow): (1) Target Discovery → TargetProfile; (2) Environment Profiling (applicable analyzers); (3) Static / dependency / configuration / binary / web analyzers (each isolated); (4) Dynamic

Analysis (if enabled and CLI entry exists); (5) Fuzzing (if enabled); (6) Finding Correlation; (7) Risk Assessment; (8) Coverage Calculation; (9) Persistence and reporting.

Analyzer failures are caught per stage; the scan continues. Progress is emitted via callback and structured logs. Cancellation is supported via CancellationToken.

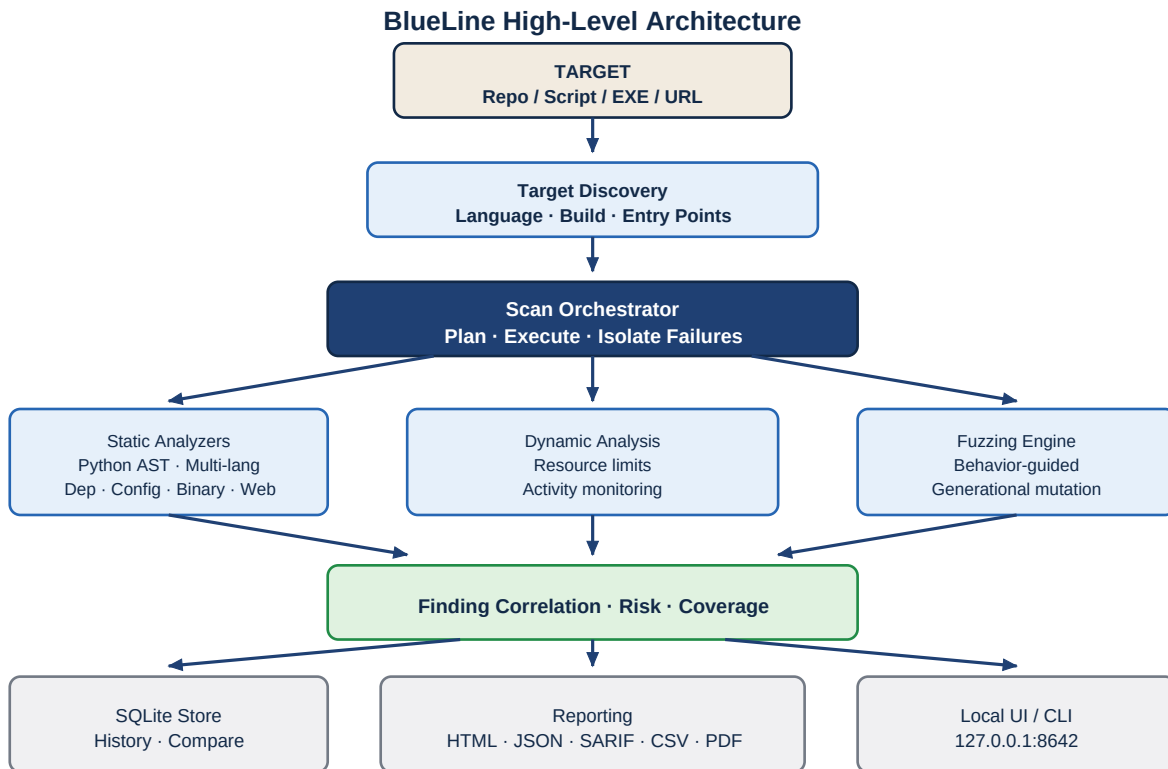


Figure 1. BlueLine high-level architecture (from repository structure and orchestrator).

Analysis Pipeline (Orchestrator Stages)



Each stage is isolated — one failure does not abort the scan.

Dynamic / Fuzz skipped when disabled or no CLI entry point.

Figure 2. Analysis pipeline stages as implemented in ScanOrchestrator.

8. Target Discovery and Input Handling

TargetDiscovery inspects a path (or URL for web) and builds a TargetProfile: languages, build systems, package managers, entry points, config files, interfaces (e.g., CLI, HTTP API), executables, file counts, size, and a list of AnalyzerCapability declarations. Exclude globs (node_modules, .git, venv, etc.) are applied. Unsupported surfaces are recorded rather than silently omitted. The orchestrator then asks the registry which analyzers applies_to the profile.

Supported target kinds in practice: source trees, scripts (especially Python), binaries (ELF/PE/Mach-O), and live HTTP targets for the web analyzer.

9. Static Analysis

9.1 Python (FULL_SUPPORT)

Implemented in `python_static.py` using the standard-library `ast` module. A `NodeVisitor` walks the tree. Rules include (non-exhaustive): `eval/exec`, `os.system/popen`, `subprocess` with `shell=True`, `pickle load`, `yaml.load` without `SafeLoader`, weak hashes (`md5/sha1`), hardcoded credential-shaped literals, dynamically built SQL (f-strings, concatenation, `format`) including same-function taint via a set of variables assigned dynamic SQL expressions, `tempfile.mktemp`, `tarfile.extractall` without filters, insecure `chmod`, Flask `debug/bind-all`, `assert` used in auth-like contexts, bare `except`. Each rule maps to title, category, severity, confidence, validation status, impact, remediation, and RiskFactors. Snippets and locations are attached as Evidence.

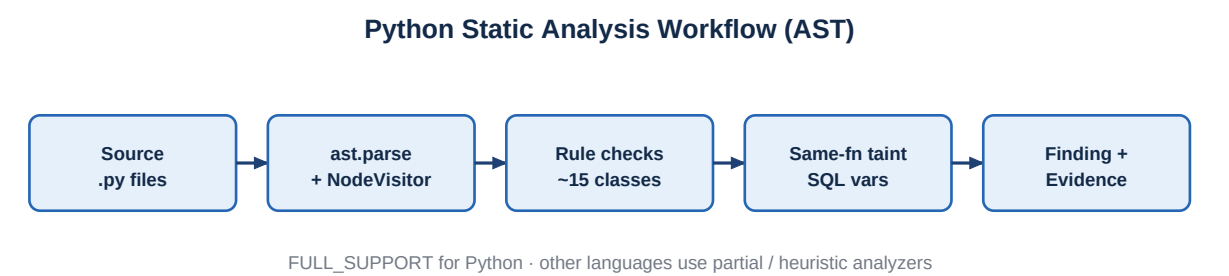


Figure 3. Python static-analysis workflow (AST-based).

9.2 Other Languages and Binary/Config

Separate modules (`js_static`, `java_static`, `go_static`, `ruby_static`, `php_static`, `rust_static`, `kotlin_static`, `swift_static`, `scala_static`) implement lighter, typically pattern- or limited-parse based checks. C/C++ is experimental. Dependency and configuration analyzers inspect package manifests, `.env`-style secrets, Dockerfiles, etc. Binary analysis: ELF parser (struct-based) extracts architecture, PIE, stripped status, dynamic dependencies; PE extracts machine/subsystem/sections/timestamp (no full import table); Mach-O primarily string extraction.

10. Multi-Language Support

Capability levels are declared per analyzer and surfaced in the profile and coverage. Unsupported languages/targets are reported, not hidden.

Language / Target	Capability
Python	FULL_SUPPORT (AST + same-function taint)
JavaScript / TypeScript	PARTIAL_SUPPORT
Java	PARTIAL_SUPPORT
Go	PARTIAL_SUPPORT
Ruby	PARTIAL_SUPPORT
PHP	PARTIAL_SUPPORT

Language / Target	Capability
Rust	PARTIAL_SUPPORT
Kotlin	PARTIAL_SUPPORT
Swift	PARTIAL_SUPPORT
Scala	PARTIAL_SUPPORT
C / C++	EXPERIMENTAL
ELF / PE / Mach-O	PARTIAL_SUPPORT
Web / API	PARTIAL_SUPPORT

Table 1. Language and target capability matrix (repository evidence).

11. Dynamic Analysis

DynamicRunner executes a command list under timeout, with process-group isolation and optional resource limits. Observations include exit code, stdout/stderr, duration, classification (normal / crash / timeout / spawn_error), and optional file/network activity via psutil polling. The orchestrator runs a fixed set of interesting inputs (empty, zeros, long strings, nulls, format strings, path traversal probes) up to dynamic_max_runs. Crashes/timeouts become findings; observed network endpoints can produce additional findings. Dynamic analysis is skipped if disabled by profile or no invocable CLI entry point is found.

Limitations: no deep instrumentation, no guaranteed OS sandbox beyond process isolation, stdin-centric input delivery.

12. Behavior-Guided Generational Fuzzing

FuzzingEngine + Mutator implement a generational loop. Generation 0 produces a batch of mutated seeds (boundaries, weird strings, etc.). Each case is executed via DynamicRunner. Anomalies (crash/timeout/spawn_error) are deduplicated by classification + exit code + first stderr line; representative input bytes are retained for reproduction. A behavior signature (classification, exit code, stdout/stderr prefixes) identifies “interesting” inputs even if non-crashing. Interesting inputs seed the next generation (capped); stagnation for two generations stops growth. Time and case budgets are enforced. Mutator uses a fixed seed (1337) by default for reproducibility.

Tests (test_fuzzing_generational.py) compare flat vs generational strategies on a staged multi-condition bug and document improved detection for the generational approach. This is not edge-coverage-guided fuzzing (no target instrumentation).

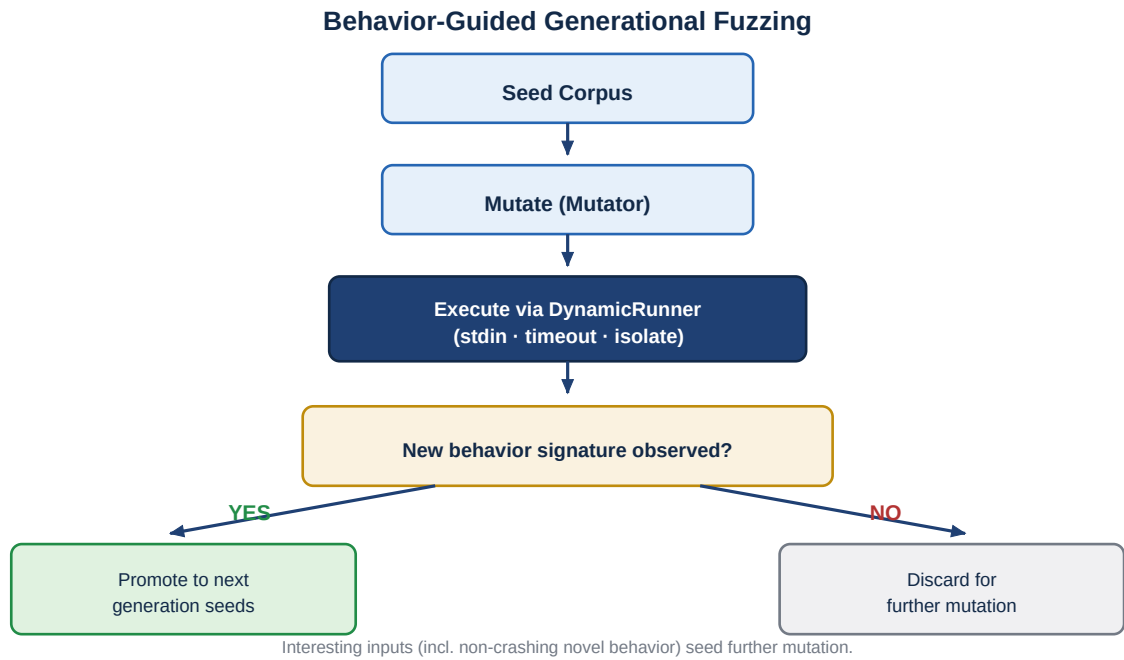


Figure 4. Behavior-guided generational fuzzing workflow.

13. Finding Correlation and Validation

Findings share a fingerprint derived from category | subcategory | affected_component | location. The correlator groups by fingerprint, keeps the highest-confidence primary, merges detector_sources and evidence, and elevates validation status if any member is CONFIRMED (or PROBABLE). Title may note corroboration by multiple detectors.

ValidationStatus values: CONFIRMED, PROBABLE, SUSPECTED, UNVERIFIED, FALSE_POSITIVE. Dynamic/fuzz crashes typically produce CONFIRMED; strong static patterns produce PROBABLE; weaker heuristics SUSPECTED.

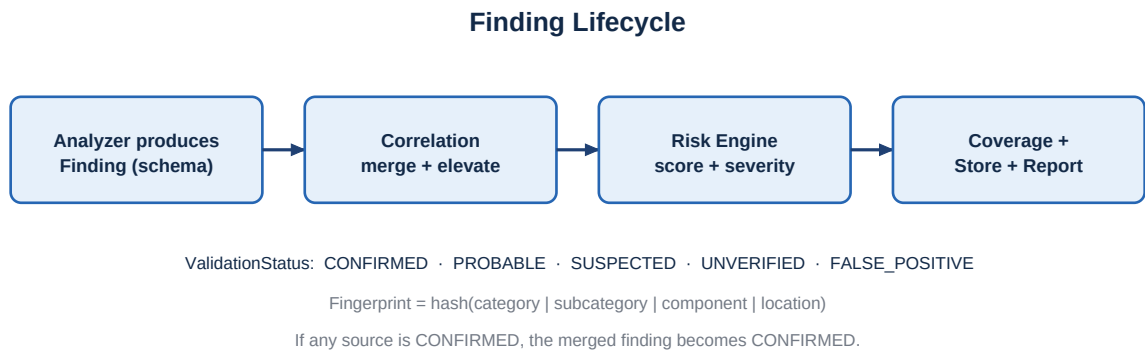


Figure 5. Finding lifecycle from production through correlation, risk, and reporting.

14. Risk Assessment

RiskEngine applies a documented formula:

$$\text{risk_score} = (0.35 \cdot \text{impact_norm} + 0.25 \cdot \text{exploitability_norm} + 0.15 \cdot \text{exposure_norm} + 0.15 \cdot \text{confidence_norm} + 0.10 \cdot \text{reproducibility_norm}) \times 100$$

where impact/exploitability/exposure/reproducibility are normalized from 0–10 and confidence from 0–100. Severity bands are derived from the score (e.g., ≥ 85 CRITICAL, ≥ 65 HIGH, ≥ 40 MEDIUM, ≥ 15 LOW, else INFORMATIONAL). Analyzers may supply RiskFactors; the engine can recompute severity for consistency. Severity, confidence, and risk_score remain distinct axes.

15. Coverage

CoverageEngine maps each stage result to a coverage percentage: completed $\approx 100 - \text{coverage_impact}$, partial lower, failed 0, skipped null. Overall assessment coverage is the average of measured stages. Unsupported capabilities from the profile are listed. Explicit note in output: high coverage means applicable analyzers ran successfully over the applicable surface; it does not mean the target is free of vulnerabilities outside that surface.

16. Persistence and History

ScanStore uses SQLite (blueline_data.sqlite3 under user data paths). Schema stores scan metadata (path, profile, status, timestamps, severity counts, overall coverage) plus full result_json. Thread-safe via lock. Supports save, load, delete, delete_all, list_history, and compare (new / resolved / worsened / unchanged findings by fingerprint). Enables regression comparison between scans.

17. Logging and Observability

Five separate rotating JSONL streams under the user data logs directory:

- **app** — process lifecycle
- **scanner** — orchestrator stage transitions
- **analyzer** — analyzer failures/warnings
- **security** — HIGH/CRITICAL finding metadata only (no evidence snippets)
- **runtime** — dynamic/fuzz execution events

Entries are structured JSON lines. Evidence/raw target content is deliberately excluded from logs to limit sensitive leakage.

18. Reporting System

Format	Purpose
HTML	Human-readable report with chart support
JSON	Full ScanResult serialization for machine consumption
SARIF 2.1.0	CI / developer tool integration; rules, locations, property bag
CSV	Tabular findings for spreadsheets
PDF	Generated when dependencies allow; API returns 501 on failure

Table 2. Implemented report formats.

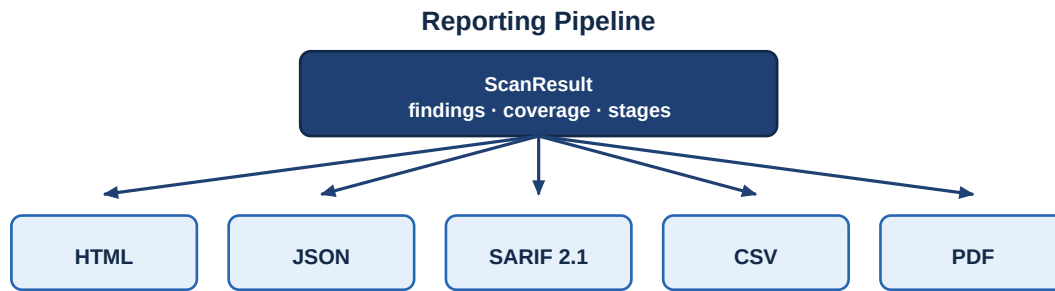


Figure 6. Reporting pipeline from ScanResult to export formats.

19. User Interface

Flask application serves `ui/index.html` and static assets. API endpoints: target discover, scan start/status/cancel/findings/report, history (GET/DELETE), single-scan delete, compare. Scans run in background threads; UI polls status. Bound exclusively to `127.0.0.1:8642`. Desktop launcher and CLI provide alternative entry points. Accessibility-related tests exist (`test_accessibility.py`).

20. Privacy and Offline-First Architecture

- No remote BlueLine service receives target data.
- Analysis, SQLite store, and logs reside under local user data paths.
- API binds to loopback only.
- Logs omit evidence snippets.
- Web analyzer is read-only (GET/HEAD/OPTIONS).

This is a design for local processing, not a formal guarantee against all host-level compromise or user-initiated export of reports.

21. Error Handling and Recovery

Orchestrator wraps each analyzer and dynamic/fuzz stage in `try/except`; failures become `StageResult(status="failed")` with error text and optional coverage impact; other stages continue. Analyzers are expected to isolate per-file errors internally. `CancellationToken` allows cooperative cancel. Malformed inputs are handled by parsers (ELF stress tests claim 0 crashes on truncated variants). PDF generation surfaces a dedicated error type.

22. Testing and Verification

The `tests/` directory contains a large set of focused modules, including:

- Language analyzers: `test_python_analyzer`, `test_js_c_and_reports`, `test_java_go_analyzers`, `test_ruby_php_analyzers`, `test_rust_analyzer`, `test_kotlin_swift_scala_analyzers`
- Binary: `test_binary_analysis`, `test_elf_parser`, `test_pe_parser`
- Dependency / web: `test_dependency_analyzer`, `test_web_api_analyzer`
- Dynamic / fuzzing: `test_dynamic_and_fuzzing`, `test_fuzzing_generational`

- Core: test_risk_coverage_correlation, test_orchestrator_isolation, test_discovery
- Persistence / logging / API: test_persistence, test_logging, test_api_server
- Reports / packaging: test_pdf_report, test_pyinstaller_spec, test_build_windows_bat
- Other: test_self_scan, test_benchmark, test_paths, test_accessibility

Tests use the standard library unittest (pytest also works). They validate rule detection, parser correctness against ground truth, generational vs flat fuzzing behavior, isolation of broken analyzers, report formats, persistence, and packaging artifacts. Exact total count is not hard-coded as a single number; the suite is organized by subsystem.

23. Build and Deployment

- Runtime: Python 3 with dependencies in requirements.txt (flask, click, psutil; minimal).
- Packaging: packaging/blueline.spec (PyInstaller), build_windows.bat, requirements-packaging.txt.
- Can be run from source (CLI / API server) or packaged as a Windows executable/bundle.
- User data (DB, logs) resolved via app/core/paths.py.

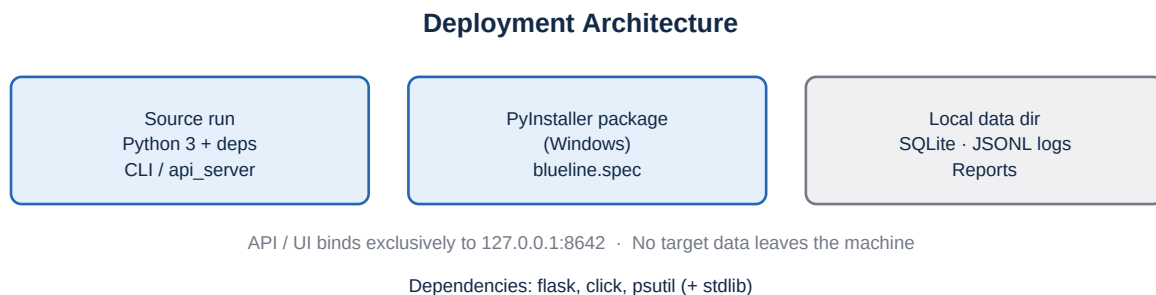


Figure 7. Deployment architecture (source, packaging, local data).

24. Security Considerations

- Local execution of untrusted targets under resource limits and process isolation; residual risk of host impact remains.
- Parsers (especially binary) must be robust; ELF path includes corruption stress testing.
- UI/API intentionally loopback-only; still requires host access control.
- Logs avoid embedding secrets/source snippets.
- Subprocess and temporary-file handling require care; dynamic/fuzz inputs can be adversarial.
- No claim of a full sandbox or formal isolation boundary beyond the implemented controls.

25. Technical Challenges

- Honest multi-language capability without over-claiming.
- Same-function taint vs true interprocedural analysis.
- Behavior-guided (not coverage-guided) fuzzing on arbitrary binaries/scripts.
- Failure isolation across heterogeneous analyzers.
- Transparent risk and coverage that avoid a false sense of security.
- Packaging a multi-component Python tool with UI and native binary handling.
- Thread-safe local SQLite under concurrent API + scan workers.

26. Design Decisions and Trade-offs

Decision	Reason	Benefit	Trade-off
Offline / localhost-only	Privacy and simplicity	Target data stays local	No remote collaboration
Unified Finding schema	Correlation & reporting	Consistent pipeline	Analyzers must conform
Per-stage isolation	Resilience	One failure does not abort scan	Partial results need clear coverage
AST for Python only (deep)	Depth where stdlib AST available	High-quality Python findings	Other languages shallower
Behavior-guided fuzzing	Multi-condition bugs without instrumentation	Demonstrable improvement over flat	Not coverage-guided
SQLite + JSON blob	Simple local history	No extra DB dependency	Not a multi-tenant schema
Transparent risk formula	Auditability	Visible factor contributions	Heuristic, not ground truth
SARIF 2.1 export	CI/tooling interoperability	Standard format	BlueLine-specific properties
Flask local UI	Rapid iteration	Single process, easy packaging	Not a full native desktop UI

Table 3. Major design decisions and trade-offs.

27. Current Limitations

- Partial/experimental language support; no deep interprocedural analysis.
- Dynamic/fuzzing limited to stdin-driven CLI-like entry points.
- Web analyzer is intentionally non-exploitative.
- PE/Mach-O analysis incomplete relative to ELF.
- Fuzzing is not coverage-guided.
- PDF generation depends on optional dependencies.
- No formal sandbox beyond process limits.
- Confidence and validation remain heuristic except when dynamic/fuzz evidence elevates them.
- Platform packaging focus visible primarily on Windows scripts; broader cross-platform packaging maturity varies.

28. Future Development

Potential extensions (not present in current code):

- Deeper interprocedural and cross-file analysis for Python and selected languages.
- Expanded binary analysis (imports, more PE/Mach-O structure).
- Optional sandboxing layers.
- Coverage-guided fuzzing when build artifacts allow instrumentation.
- Richer web checks still constrained to non-destructive methods.
- CI/CD plugins consuming SARIF.
- Additional report consumers and policy engines.
- Improved corpus management and differential fuzzing.

29. Conclusion

BlueLine implements a coherent offline pipeline that discovers targets, runs only applicable techniques, correlates and scores findings under an explicit multi-axis model (severity / confidence / validation / risk / coverage), and reports both results and assessment coverage. Python receives the strongest static analysis; other languages and binary/web surfaces are handled at declared partial or experimental levels. Dynamic execution and behavior-guided generational fuzzing supply runtime evidence when entry points exist. Local SQLite history, structured logging, and multi-format reports (including SARIF 2.1) complete the loop.

The system is mature enough as a research/engineering platform and portfolio artifact to demonstrate real multi-technique design, failure isolation, and honest capability reporting, while remaining explicit about its current limits.

30. Technical Appendix

30.1 Repository Structure (Top-Level)

app/ (analyzers, core, fuzzing, persistence, reporting, runtime, targets, api_server, cli, desktop_launcher), ui/, tests/, packaging/, docs/, test-targets/, requirements.txt, LICENSE.md, README.md.

30.2 Key Classes / Components

- ScanOrchestrator, Analyzer / AnalyzerRegistry
- Finding, TargetProfile, ScanConfig, ScanResult, StageResult
- FindingCorrelator, RiskEngine, CoverageEngine
- DynamicRunner, FuzzingEngine, Mutator
- ScanStore, TargetDiscovery

30.3 Dependencies

Runtime: flask ≥ 3.0, click ≥ 8.1, psutil ≥ 5.9 (+ standard library: ast, sqlite3, struct, subprocess, logging, etc.). Packaging extras are listed separately under packaging/.

30.4 Scan Profiles

Profile	Dynamic	Fuzzing	Notes
quick	Off	Off	Static-focused
standard	On (limited runs)	Off	Default
deep	On	On (higher budget)	More thorough
maximum	On	On (high budget)	Longer campaigns

Table 4. Scan configuration profiles.

30.5 Default API Binding

127.0.0.1:8642 (loopback only).

30.6 Representative Risk Formula (from source)

```
risk_score = (  
    0.35 * impact_norm +  
    0.25 * exploitability_norm +  
    0.15 * exposure_norm +  
    0.15 * confidence_norm +  
    0.10 * reproducibility_norm  
) * 100
```

31. References

- [1] OASIS SARIF 2.1.0 specification (schema referenced in `generate_sarif_report`).
- [2] Python standard library documentation (`ast`, `sqlite3`, `struct`, `subprocess`, `logging`).
- [3] Flask documentation.
- [4] `psutil` documentation.
- [5] Project repository: <https://github.com/AmirSam-Eftekhari/BlueLine>

End of document. All technical claims are grounded in the inspected source code of the BlueLine repository. Features not present in the repository are not claimed.